

Wongnok Recipe API

Data Flow: Favorite & Rating

สรุป flow การไหลของข้อมูล ทีละไฟล์ ทีละบรรทัด
สำหรับใช้ทบทวนก่อนสอบกลางภาค (Golang Day 9)

#	หัวข้อ
1	Favorite (POST /recipes/:id/favorite)
2	Unfavorite (DELETE /recipes/:id/favorite)
3	Query Favorite (GET /recipes?favorite=)
4	POST Rating (POST /recipes/:id/rating)
5	averageRating (คำนวณ + แสดงผล)

โครงสร้างทุกพีเจียร์เดินตาม Layer เดียวกัน: **handler.go** → **service.go** → **repository.go**

1) Favorite — POST /recipes/:id/favorite

handler.go (บรรทัด 182-205)

บรรทัด 183 reqctx.UserID(ctx.Request.Context()) — ดึง userID จาก token ของคนที่ล็อกอิน (middleware JWT เก็บไว้ให้ก่อนหน้านี้แล้ว)

บรรทัด 184-187 ถ้าไม่มี userID (ไม่ได้ login) → ตอบ 401 ทันที

บรรทัด 189 strconv.Atoi(ctx.Param("id")) — แปลง :id จาก URL (string) เป็นเลข recipeID

บรรทัด 195 hdr.service.Favorite(ctx.Request.Context(), userID, recipeID) — จุดส่งไม้ต่อให้ชั้น Service ทำงาน

บรรทัด 204 ถ้าไม่มี error → ตอบ 204 No Content

service.go (บรรทัด 143-149)

บรรทัด 145 svc.FindByID(ctx, recipeID) — เช็คก่อนว่าสูตรอาหาร id นี้มีอยู่จริงไหม

บรรทัด 146 ถ้าไม่เจอ → return ErrRecipeNotFound กลับไปให้ handler แปลงเป็น 404

บรรทัด 148 svc.repository.Favorite(ctx, userID, recipeID) — ส่งไม้ต่อให้ชั้น Repository ไปเขียนข้อมูลจริง

repository.go (บรรทัด 150-162) — จุดเขียนข้อมูลจริงลง DB

```
func (repo *repository) Favorite(ctx context.Context, userID uuid.UUID, recipeID int) error {
    // กับดัก T1: ใช้ ON CONFLICT เพื่อคืนชีพแถวข้อมูล
    return repo.db.WithContext(ctx).Clauses(clause.OnConflict{
        Columns: []clause.Column{{Name: "user_id"}, {Name: "recipe_id"}},
        DoUpdates: clause.Assignments(map[string]interface{}{
            "deleted_at": nil,
            "updated_at": time.Now(),
        }),
    }).Create(&UserFavorite{
        UserID: userID,
        RecipeID: recipeID,
    }).Error
}
```

เขียนเข้าตาราง user_favorites — พยายาม INSERT ก่อน ถ้าเจอว่ามีแถวนี้อยู่แล้ว (เคยกด favorite แล้ว unfavorite ไป) ON CONFLICT จะสั่งให้ UPDATE แทน คือคืนชีพแถวเดิม (deleted_at = nil) แทนที่จะ error ว่าซ้ำ

Request → handler.go:183 (เช็ค login / ดึง userID)
→ handler.go:189 (แปลง id)
→ handler.go:195 → ส่งต่อ service
→ service.go:145 (เช็คสูตรมีจริง)

- service.go:148 → ส่งต่อ repository
- repository.go:152-161 (เขียนจริงลง DB ตาราง user_favorites)
- กลับมา handler.go:204 (ตอบ 204)

2) Unfavorite — DELETE /recipes/:id/favorite

handler.go (บรรทัด 220-243)

บรรทัด 221 ดึง userID จาก token (เหมือน Favorite เป๊ะ)

บรรทัด 227 แปลง :id เป็น recipeID

บรรทัด 233 `hdr.service.Unfavorite(...)` → ส่งต่อ service

service.go (บรรทัด 151-156)

บรรทัด 152 เช็คสูตรมีอยู่จริงไหม (`svc.FindByID`)

บรรทัด 155 `svc.repository.Unfavorite(...)` → ส่งต่อ repository

repository.go (บรรทัด 164-169) — จุดเขียนจริง

```
func (repo *repository) Unfavorite(ctx context.Context, userID uuid.UUID, recipeID int) error {
    // การลบแบบ Soft Delete อาศัย gorm.DeletedAt ใน Struct
    return repo.db.WithContext(ctx).
        Where("user_id = ? AND recipe_id = ?", userID, recipeID).
        Delete(&UserFavorite{}).Error
}
```

ทำ `UPDATE user_favorites SET deleted_at = now() WHERE user_id = ? AND recipe_id = ?` (soft delete เพราะ struct `UserFavorite` มี `gorm.DeletedAt`) — ต่างจาก Favorite ตรงที่ใช้ `Delete` ตรงๆ ไม่ต้องกังวลเรื่องซ้ำ

Request → handler.go:221 (เช็ค login / ดึง userID)
→ handler.go:227 (แปลง id)
→ handler.go:233 → ส่งต่อ service
→ service.go:152 (เช็คสูตรมีจริง)
→ service.go:155 → ส่งต่อ repository
→ repository.go:166-168 (soft delete ตาราง user_favorites)
→ กลับมา handler.go:242 (ตอบ 204)

3) Query Favorite — GET /recipes?favorite=true|false

dto.go (บรรทัด 93-99) — จดรับค่า

```
type GetRecipesQuery struct {
    Pagination
    Name      string      `form:"name"`
    Difficulty string      `form:"difficulty"`
    Sort      SortDirection `form:"sort" binding:"omitempty,oneof=ASC DESC"`
    Favorite  *bool       `form:"favorite"`
}
```

*bool (pointer) ไม่ใช่ bool ธรรมดา เพื่อแยก 3 สถานะ: ไม่ส่ง favorite มาเลย (nil) / ส่ง true / ส่ง false

handler.go (บรรทัด 140-167) — GetRecipes

บรรทัด 149 `ctx.ShouldBindQuery(&query)` — แปลง URL query string → struct

บรรทัด 155 `hdr.service.List(...)` → ส่งต่อ service

service.go (บรรทัด 66-92) — List

บรรทัด 81 `svc.repository.List(...)` → ส่งต่อ repository (ก่อนหน้านี้แค่เช็ค default value/difficulty)

repository.go (บรรทัด 92-138) — List — จุดกรองข้อมูลจริง

```
// กับดัก T3: ใช้ Subquery เพื่อกรอง Favorite ไม่ใช่ JOIN
if query.Favorite != nil {
    subQuery := repo.db.WithContext(ctx).Model(&UserFavorite{}).
        Select("recipe_id").Where("user_id = ?", userID)
    if *query.Favorite {
        db = db.Where("recipes.id IN (?)", subQuery)
    } else {
        db = db.Where("recipes.id NOT IN (?)", subQuery)
    }
}
```

บรรทัด 104 สร้าง subquery หา recipe_id ทั้งหมดที่ user คนนี้เคยกด favorite ไว้ (จากตาราง `user_favorites`) แล้ว บรรทัด 106/108 เอาไปกรอง `recipes` หลักด้วย `IN / NOT IN`

ใช้ **subquery** แทน **JOIN** เพื่อไม่ให้ recipe ที่มีคนกด favorite ซ้ำหลายคน (ในอนาคต) โดน join ซ้ำเป็นหลายแถว

Request → dto.go:98 (นิยาม field รับค่า favorite)
→ handler.go:149 (bind query string → struct)
→ handler.go:155 → ส่งต่อ service
→ service.go:81 → ส่งต่อ repository

- repository.go:103-110 (สร้าง subquery + WHERE IN/NOT IN)
- repository.go:132 (Find ข้อมูลที่กรอกแล้ว)
- กลับมา handler.go:166 (ตอบ 200 พร้อมผลลัพธ์)

4) POST Rating — POST /recipes/:id/rating

dto.go (บรรทัด 227)

```
type RateRecipeRequest struct {  
    Score float64 `json:"rating" binding:"required,min=1,max=5"`  
}
```

handler.go (บรรทัด 260-289) — Rate

บรรทัด 261 ดึง userID

บรรทัด 267 แปลง :id

บรรทัด 273-277 ctx.ShouldBindJSON(&req) — bind body JSON {"rating": 4} เข้า req.Score

บรรทัด 279 hdr.service.Rate(ctx.Request.Context(), userID, recipeID, req.Score) → ส่งต่อ service

service.go (บรรทัด 158-163) — Rate

บรรทัด 159 เช็คว่าสูตรมีจริงไหม

บรรทัด 162 svc.repository.Rate(...) → ส่งต่อ repository

repository.go (บรรทัด 171-207) — Rate — ขั้นที่ 1: เขียนคะแนน

```
err := tx.Clauses(clause.OnConflict{  
    Columns: []clause.Column{{Name: "user_id"}, {Name: "recipe_id"}},  
    DoUpdates: clause.Assignments(map[string]interface{}{  
        "score": score,  
        "deleted_at": nil,  
        "updated_at": time.Now(),  
    }),  
}).Create(&RecipeRating{  
    UserID: userID,  
    RecipeID: recipeID,  
    Score: score,  
}).Error
```

เขียนลงตาราง `recipe_ratings` — คนเดิมให้คะแนนซ้ำ = แทนที่คะแนนเก่าด้วย `OnConflict` (เหมือน trap T1 ของ Favorite) ทำงานอยู่ใน **transaction เดียวกัน** กับขั้นตอนคำนวณ `averageRating` (หวั่นข้อถัดไป)

Request → handler.go:261 (เช็ค login / ดึง userID)
→ handler.go:267 (แปลง id)
→ handler.go:273-277 (bind body → req.Score)

- handler.go:279 → ส่งต่อ service
- service.go:159 (เช็คค่าสูตรมีจริง)
- service.go:162 → ส่งต่อ repository
- repository.go:176-190 (เขียนคะแนนลง recipe_ratings)
- repository.go:192-205 (คำนวณ + เขียน averageRating – ดูหัวข้อ 5)
- กลับมา handler.go:288 (ตอบ 204)

5) averageRating — คำนวณ + แสดงผล

คำนวณที่ repository.go (บรรทัด 171-207) — Rate — ขั้นที่ 2-3 (ต่อจากหัวข้อ 4)

```
// ขั้น 2 (บรรทัด 192-195) — คำนวณค่าเฉลี่ย
var avgScore float64
err = tx.Model(&RecipeRating{}).
    Select("COALESCE(AVG(score), 0)").
    Where("recipe_id = ? AND deleted_at IS NULL", recipeID).
    Scan(&avgScore).Error

// ขั้น 3 (บรรทัด 202-205) — เขียนค่าเฉลี่ยลง recipes
return tx.Model(&Recipe{}).
    Where("id = ?", recipeID).
    Update("average_rating", avgScore).Error
```

`COALESCE(AVG(score), 0)` คำนวณค่าเฉลี่ยจากทุกคะแนนของสูตรนี้ (ทุก user ที่เคยให้ไว้) ครอบด้วย `COALESCE` กัน ปัญหา `AVG()` คิ่ `NULL` ตอนยังไม่มีใครให้คะแนนเลย (scan `NULL` เข้า `float64` ตรงๆ จะ error) แล้วเขียนทับเก็บไว้ที่ column `recipes.average_rating` ล่วงหน้า ไม่ต้องคำนวณสดทุกครั้งตอนอ่าน

ทำไมต้องอยู่ใน transaction เดียวกัน

เพื่อกันเคสที่ขึ้นบันทึกคะแนนสำเร็จ แต่ขั้นเขียน `averageRating` ล้มเหลว ซึ่งจะทำให้ข้อมูลไม่ตรงกัน (คะแนนบันทึกลแล้วแต่ค่าเฉลี่ยยังเป็นเลขเก่า) — ถ้าอยู่ใน transaction เดียวกัน พลาดจุดไหนก็ `rollback` ทั้งหมด

แสดงผลออกไปที่ response — dto.go

บรรทัด 151 `AverageRating float64 `json:"averageRating"`` — field ใน `RecipeResponse` ตามที่โจทย์สั่ง

บรรทัด 157, 198 ฟังก์ชัน `NewRecipeResponse` → `AverageRating: recipe.AverageRating` — ตอนแปลง `Recipe` (จาก DB) เป็น `RecipeResponse` (JSON ที่ส่งกลับ) หยิบค่าที่คำนวณ+บันทึกลงมาแล้วมาใส่ตรงนี้เลย ไม่ได้คำนวณใหม่

เขียนค่า: `POST /rating` → `repository.go:192-205` (คำนวณ AVG แล้วเขียนลง `recipes.average_rating`)
อ่านค่า: `GET /recipes` หรือ `GET /recipes/:id`
→ `repository.go` (`FindByID` / `List`) ดึง `Recipe` ที่มี `average_rating` ติดมาด้วย
→ `dto.go:198` (`NewRecipeResponse` ใส่ค่าเข้า field `AverageRating`)
→ ตอบกลับเป็น JSON `"averageRating": ...`

สรุป: คำนวณครั้งเดียวตอน `POST /rating` ทำงาน แล้วอ่านซ้ำได้เรื่อยๆ ตอน `GET` โดยไม่ต้องคำนวณใหม่ทุกครั้ง — เร็วกว่าและลดภาระ DB